A Critical Review of Recurrent Neural Networks for Sequence Learning

系列学習のための再帰型ニューラルネットワークの批判的総説

- 著者
  - ザカリー・C・リプトン（Zachary C. Lipton）
  - ジョン・バーコウィッツ（John Berkowitz）
  - チャールズ・エルカン（Charles Elkan）
- 掲載
  - arXiv:1506.00019 [cs.LG]（プレプリント、査読なし）
  - 使用した版：v4（2015年10月17日）。本文に記載された日付は2015年6月5日
- 用語
  - 本稿の「再帰型ニューラルネットワーク（RNN）」は recurrent neural network（時間方向に再帰する）を指す。木構造に沿って計算する recursive neural network とは別物である
  - 📍 p.1 表紙・左端の arXiv 表記、p.25 第3段落

Abstract

- 無数の学習タスクでは、系列データ（sequential data）を扱う必要がある。
  - モデルが系列を出力する必要があるタスク
    - 画像キャプションの生成
    - 音声合成
    - 音楽生成
  - モデルが系列の入力から学習する必要があるタスク
    - 時系列予測
    - 動画解析
    - 音楽情報検索
  - 対話型（interactive）のタスクは、多くの場合この両方を必要とする
    - 自然言語の翻訳
    - 対話
    - ロボットの制御
  - 📍 p.1 Abstract 1–7行目
- 再帰型ニューラルネットワーク（RNN）
  - ノード間のネットワーク内にある循環（ループ）によって、系列のダイナミクスを捉えるコネクショニストモデル（ニューラルネットワーク型のモデル）である。
  - 標準的な順伝播型ニューラルネットワークとは異なり、任意の長さのコンテキストウィンドウの情報を表現できる状態を保持する。
  - RNN は従来は訓練が難しく、多くの場合は数百万のパラメータを含む。それでも次の進歩によって、RNN を用いた大規模学習が成功するようになった。
    - ネットワークアーキテクチャの進歩
    - 最適化手法の進歩
    - 並列計算の進歩
  - 📍 p.1 Abstract 7–12行目

- 近年は、長短期記憶（long short-term memory; LSTM）や双方向 RNN（bidirectional RNN; BRNN）に基づくシステムが、次のようなタスクで画期的な性能を示している。
  - 画像キャプションの生成
  - 言語翻訳
  - 手書き文字認識
  - 📍 p.1 Abstract 12–15行目
- 本稿では、この強力な学習モデルを過去30年で生み出し、実用化した研究を概観し、統合する。
- 必要に応じて、互いに食い違う表記法や用語を整理する。
- 本稿の目的：最先端の手法について、歴史的な視点と一次研究への参考文献を備えた自己完結的な説明を提供すること
  - 📍 p.1 Abstract 15–20行目

1 イントロダクション

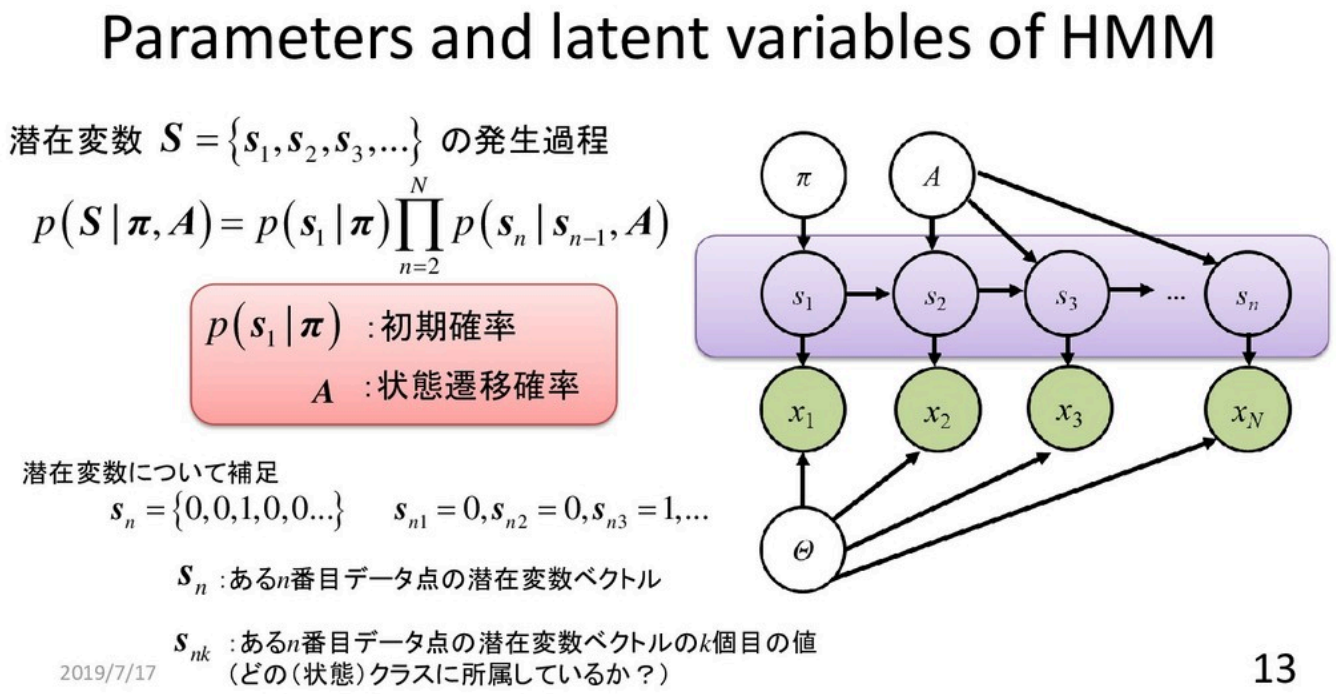
- ニューラルネットワーク
  - 強力な学習モデルであり、教師あり学習・教師なし学習の幅広いタスクで最先端の成果を達成している。
  - 生の特徴量が個別には解釈できない機械知覚のタスクに特に適している。その理由は、人手で設計した特徴量に頼らず、階層的な表現を学習できることにある。
  - ストレージの低価格化、データセットの大規模化、並列計算の進歩によって、深層学習（深層信念ネットワーク、畳み込みニューラルネットワーク）が多くの応用で記録的な成果を挙げている。
  - 📍 p.1 §1 第1段落 1–2行目、p.2 第1段落
- 標準的なニューラルネットワークの限界
  - 訓練例とテスト例が互いに独立していることを前提にしている。1つの例を処理するたびに、ネットワークの状態はすべて失われる。
  - データが時間や空間で関係しあう場合、この前提は成り立たない（動画のフレーム、音声の断片、文中の単語）。
  - 入力が固定長のベクトルであることも前提にしている。
  - 📍 p.2 第2段落 1–10行目
- RNN が必要な理由
  - RNN は系列データを1要素ずつ処理しながら、系列のステップをまたいで情報を選択的に受け渡せる。
  - そのため、独立でない要素の系列からなる入力や出力をモデル化でき、複数の時間スケールの依存関係も同時に扱える。
  - 📍 p.2 第2段落 10–18行目
- 本稿の立場
  - RNN の研究には、認知モデリング（生物学的なもっともらしさを重視）と教師あり機械学習（実験的な性能を重視）という2つの源流がある。
  - 本稿は実験的な性能を重視する立場で書かれている。ただし必要に応じて神経科学の概念との関係にも触れる。
  - 📍 p.2 第3段落

1.1 なぜ系列性を明示的にモデル化するのか？

- サポートベクターマシン、ロジスティック回帰、順伝播型ネットワークは、時間を明示的にモデル化しなくても有用だった。独立性の仮定こそが近年の進歩をもたらしたともいえる。

- 前後の入力を連結したスライディングウィンドウを使えば、時間を暗黙的に扱える（例：音声モデリングにおける深層信念ネットワーク）。
- しかし独立性を仮定すると、長距離の依存関係をモデル化できない。
  - 例：長さ5のウィンドウで学習したモデルは、「6時刻前に見たデータは何か」という単純な問いに答えられない。
  - 対話システム、自動運転、ロボット手術のような対話型システムを作るには、系列性や時間の明示的なモデルが必要である。
- 📍 p.3 §1.1 第1–2段落

1.2 なぜマルコフモデルを使わないのか？



- マルコフモデルの限界
  - 状態を、あまり大きくない離散の状態空間 S から選ぶ必要がある。
  - 隠れマルコフモデル（HMM）の推論に使う動的計画法の計算時間は $O(|S|^2)$ で、遷移表の大きさも $|S|^2$ になる。状態数が増えると計算できなくなる。
  - 各隠れ状態は直前の状態にしか依存できない。ウィンドウを広げるために状態空間の直積をとると、状態数がウィンドウ長に対して指数的に増える。
  - 📍 p.3 §1.2、p.4 第1段落
- RNN の利点
  - RNN の状態も、現在の入力と直前の状態だけで決まる。それでも隠れ状態は、ほぼ任意の長さの過去の情報を保持できる。
  - 隠れ層のノード数を N とすると、各ノードが2値しかとらなくても 2^N 通りの状態を表現できる。実数値ならさらに多い。
  - 表現力はノード数に対して指数的に増えるが、推論と学習の計算量はたかだか2乗でしか増えない。
  - 📍 p.4 第2段落

1.3 RNNは表現力が高すぎるのではないか？

- 有限サイズの RNN（シグモイド活性化）は万能チューリング機械を模倣できる（Siegelmann and Sontag, 1991）。
- しかし表現力が高いだけでは役に立たない。C言語も任意のプログラムを表現できるが、機械学習の万能薬とはいわれない。
  - C のプログラム空間を効率よく探索する方法はなく、損失関数の勾配も計算できない。
  - 有限のデータセットに過学習するプログラムが無数に存在する。
- RNN がこの問題を起こしにくい理由
  - アーキテクチャを固定すれば端から端まで微分可能であり、勾配に基づく学習ができる。
  - サイズとアーキテクチャを固定した RNN は、実際には任意のプログラムを再現できない。
  - 重み減衰、ドロップアウト、自由度の制限といった標準的な正則化で過学習を抑えられる。
- 📍 p.4 §1.3、p.5 冒頭

1.4 先行研究との比較

- RNN の文献は初学者にとって読みにくい。
  - 図で時刻をまたぐ辺が示されていないことが多い。
  - 表記が論文ごとに異なる。例：添字がノードと時刻の両方を表す、h が活性化関数と隠れ層の両方を表す、t が時刻と目標値の両方を表す。
- 有用な既存資料として、Graves（2012）の書籍、Gers（2001）の博士論文、De Mulder et al.（2015）の言語モデル向けサーベイ、Pearlmutter（1995）の勾配計算のサーベイがある。
- 本稿は、読みやすく直感的で、表記が一貫し、適度に網羅的な RNN のサーベイを目指す。アーキテクチャ、アルゴリズム、結果に加えて、直感、歴史的な視点、他手法との比較も示す。
- 📍 p.5 §1.4 第1–2段落

2 背景

この章では、形式的な表記法を導入し、ニューラルネットワーク全般について簡単な背景を説明する。

- 📍 p.5 §2

2.1 系列（sequence）

- 入力系列を $(\boldsymbol{x}^{(1)}, \boldsymbol{x}^{(2)}, \ldots, \boldsymbol{x}^{(T)})$、目標系列を $(\boldsymbol{y}^{(1)}, \boldsymbol{y}^{(2)}, \ldots, \boldsymbol{y}^{(T)})$ と書く。各データ点は実数ベクトルである。
  - T：系列が有限の場合の最大の時刻インデックス
  - 予測値は $\hat{\boldsymbol{y}}^{(t)}$ と書く
  - 時刻は上付きの括弧で書く。ネットワークのノード番号（下付き添字）と区別するためである
- 訓練データは（入力系列, 目標系列）の組の集合である。入力と出力のどちらか一方が単一のデータ点のこともある。
- RNN は時間的な系列に限らず、遺伝子データのような時間を持たない系列にも使える。
- 時刻は、連続的な過程を等間隔でサンプリングしたもの（動画のフレーム、音声の振幅）のこともあれば、順序だけを表すこともある（自然言語）。
  - 例：「John Coltrane plays the saxophone」では $\boldsymbol{x}^{(1)} = \mathrm{John}$、$\boldsymbol{x}^{(2)} = \mathrm{Coltrane}$
- 📍 p.5 §2.1、p.6 第1–3段落

2.2 ニューラルネットワーク

- ニューラルネットワークは、人工ニューロン（ノード、ユニット）と、その間の有向辺（生物のシナプスに対応）からなる。

- 記号
  - $l_j(\cdot)$：ノード j の活性化関数（link function とも呼ばれる）。隠れ層の値 $\boldsymbol{h}$ と区別するため、h_j ではなく l_j と書く
  - $w_{jj'}$：ノード j' から j への辺の重み（「to-from」の順）。文献によっては逆順の「from-to」なので注意する
- ノード j の値は、入力ノードの値の重み付き和に活性化関数を適用して求める。

$$v_j = l_j\left(\sum_{j'} w_{jj'} \cdot v_{j'}\right)$$

- 括弧の中の重み付き和を入力活性（incoming activation）と呼び、a_j と書く。
- 主な活性化関数
  - シグモイド：$\sigma(z) = 1/(1+e^{-z})$
  - tanh：$\phi(z) = (e^z - e^{-z})/(e^z + e^{-z})$。順伝播型でよく使われ、RNN にも使われている
  - ReLU：$l_j(z) = \max(0, z)$。音声処理や物体認識で多くの深層ネットワークの性能を改善し、RNN にも使われている
- 出力層の活性化関数はタスクで決まる。
  - K クラスの多クラス分類：softmax

$$\hat{y}_k = \frac{e^{a_k}}{\sum_{k'=1}^{K} e^{a_{k'}}} \quad (k = 1, \ldots, K)$$

  - 分母は分子の総和なので、全出力の和は1になる。
  - マルチラベル分類：要素ごとのシグモイド
  - 回帰：線形出力
- 📍 p.6 §2.2、p.7 図1・第1–4段落

2.3 順伝播型ネットワークと誤差逆伝播法

- 順伝播型ネットワーク
  - ノードの有向グラフに循環を持たないネットワーク。循環がないのでノードを層に分けられ、下の層から順に出力を計算できる。
  - 入力 $\boldsymbol{x}$ を最下層に与え、最上層で出力 $\hat{\boldsymbol{y}}$ を得る。
  - 学習では、出力 $\hat{\boldsymbol{y}}$ と目標 $\boldsymbol{y}$ の距離を罰する損失関数 $\mathcal{L}(\hat{\boldsymbol{y}}, \boldsymbol{y})$ が小さくなるように重みを更新する。
  - 📍 p.8 図2、§2.3 第1–2段落
- 誤差逆伝播法（Rumelhart et al., 1985）
  - 連鎖律で、損失 $\mathcal{L}$ の各パラメータについての微分を計算し、勾配降下法で重みを更新する。
  - 損失曲面は非凸なので、大域的最小値に到達する保証はない。厳密な最適化は NP 困難である。それでも事前学習や最適化の工夫によって、多くの教師あり学習で成功している（例：畳み込みニューラルネットワーク）。
  - 📍 p.8 第3段落、p.9 第1段落

- 確率的勾配降下法（SGD）
  - 現在はミニバッチを使う SGD で学習するのが一般的である。バッチサイズ1の更新式は次のとおり。

$$\boldsymbol{w} \leftarrow \boldsymbol{w} - \eta \nabla_{\boldsymbol{w}} F_i$$

  - ここで、η は学習率、$\nabla_{\boldsymbol{w}} F_i$ は1つの例 (x_i, y_i) で計算した目的関数の勾配である。
  - 変種：AdaGrad（勾配の2乗和で学習率を調整。学習率が単調に減る）、RMSprop（2乗和を移動平均に変えた AdaGrad）、AdaDelta、モーメンタム法（過去の更新の減衰和を加える）
  - 📍 p.9 第2段落
- 逆伝播の計算
  - 順伝播で各ノードの値 v_j と出力 $\hat{\boldsymbol{y}}$ を求め、各出力ノード k で損失 $\mathcal{L}(\hat{y}_k, y_k)$ を計算する。
  - 出力ノード k の誤差を計算する。

$$\delta_k = \frac{\partial \mathcal{L}(\hat{y}_k, y_k)}{\partial \hat{y}_k} \cdot l_k'(a_k)$$

  - 1つ下の層のノード j の誤差を計算し、これを下の層へ順に繰り返す。

$$\delta_j = l'(a_j) \sum_k \delta_k \cdot w_{kj}$$

  - δ_j は、ノード j の入力活性についての損失の微分 $\partial \mathcal{L} / \partial a_j$ を表す。
  - 重み $w_{jj'}$ についての勾配は次のようになる。

$$\frac{\partial \mathcal{L}}{\partial w_{jj'}} = \delta_j v_{j'}$$

  - 📍 p.9 第3段落、p.10 第1段落
- その他の話題
  - 遺伝的アルゴリズムによる学習が1990年代に提唱された。近年は、逆伝播では扱えないアーキテクチャ（層数、各層のノード数、層間の結合、活性化関数）の探索に使われている。
  - 学習でスパース性をどう活かすかは未解決である。シグモイドや tanh のノードは厳密に0にならないが、ReLU は隠れ層にスパース性をもたらす。
  - 📍 p.10 第2–3段落

3 再帰型ニューラルネットワーク

第1段落（RNN の定義）

- RNN は、隣り合う時刻をつなぐ辺を順伝播型ネットワークに加え、モデルに時間の概念を持ち込んだものである。
- 通常の辺どうしでは、順伝播型と同じく循環を作ってはいけない。隣り合う時刻をつなぐ辺（再帰辺, recurrent edge）だけは循環を作ってよい。長さ1の循環、つまり時刻をまたいで自分自身につながる自己結合も含む。

- 時刻 t に再帰辺を持つノードは、現在のデータ点 $\boldsymbol{x}^{(t)}$ と、1時刻前の隠れノードの値 $\boldsymbol{h}^{(t-1)}$ の両方から入力を受ける。
- 時刻 t の出力 $\hat{\boldsymbol{y}}^{(t)}$ は、その時刻の隠れノードの値 $\boldsymbol{h}^{(t)}$ から計算する。
- そのため、時刻 $t-1$ の入力 $\boldsymbol{x}^{(t-1)}$ は、再帰結合を通じて時刻 t 以降の出力に影響しうる。
- 📍 p.10 §3 第1段落

第2段落（単純 RNN の順伝播の式）

- 単純な RNN（図3）の各時刻の順伝播は、次の2式ですべて書ける。

$$\boldsymbol{h}^{(t)} = \sigma\left(W^{\mathrm{hx}}\boldsymbol{x}^{(t)} + W^{\mathrm{hh}}\boldsymbol{h}^{(t-1)} + \boldsymbol{b}_h\right)$$

$$\hat{\boldsymbol{y}}^{(t)} = \mathrm{softmax}\left(W^{\mathrm{yh}}\boldsymbol{h}^{(t)} + \boldsymbol{b}_y\right)$$

- 記号
  - W^{hx}：入力層と隠れ層の間の通常の重み行列
  - W^{hh}：隣り合う時刻の隠れ層どうしをつなぐ再帰の重み行列
  - $\boldsymbol{b}_h$、$\boldsymbol{b}_y$：各ノードがオフセットを学習するためのバイアス
- 📍 p.10 §3 第2段落、p.11 式の直後

図3（単純な再帰型ネットワーク）

- 実線の辺では、順伝播型と同じく同じ時刻の中で活性が伝わる。破線の辺は、時刻 t のノードを次の時刻 $t+1$ のノードにつなぐ。
- 📍 p.11 図3

第3段落（時間方向の展開と BPTT）

- 図3のネットワークを時間方向に展開する（unfold）と図4になる。
- 展開すると、このネットワークは循環を持つものではなく、1時刻につき1層を持ち、全時刻で重みを共有する深いネットワークと解釈できる。
- そのため、展開したネットワークは多くの時刻にわたって誤差逆伝播法で学習できる。
- この手法を時間方向の誤差逆伝播法（backpropagation through time, BPTT）と呼ぶ。Werbos（1990）が導入し、現在よく使われる再帰型ネットワークはすべてこれを使う。
- 📍 p.11 第2段落、p.12 図4

3.1 初期の再帰型ネットワーク

第1段落（Hopfield ネットワーク）

- 再帰型ネットワークの基礎研究は1980年代に行われた。
- Hopfield（1982）は、パターン認識の能力を持つ再帰型ネットワークを提案した。
  - ノード間の重みで定義され、活性化関数は0を閾値とする単純な閾値関数である。
  - ノードの値を設定してパターンを入力し、更新規則に従ってしばらく動かすと、別のパターンが読み出される。
- 壊れたパターンから記憶したパターンを復元するのに役立ち、ボルツマンマシンやオートエンコーダの先駆けになった。
- 📍 p.11 §3.1

第2段落（Jordan ネットワーク）

- 系列の教師あり学習のための初期のアーキテクチャは Jordan（1986）が提案した（図5）。
  - 隠れ層が1層の順伝播型ネットワークに、特別なユニットを追加したものである。
  - 出力ノードの値を特別なユニットに送り、特別なユニットがその値を次の時刻の隠れノードに送る。
  - 出力が行動なら、特別なユニットによって過去の時刻にとった行動を記憶できる。
- 現代のアーキテクチャにも、出力ノードから直接値を渡す似た仕組みがある。Sutskever et al.（2014）の翻訳モデルは、文を生成するとき、各時刻に選んだ単語を次の時刻の入力として与える。
- Jordan ネットワークの特別なユニットは自己結合を持つ。この辺によって、途中の各時刻の出力を乱さずに、複数の時刻にわたって情報を送れる。
- 脚注2：Jordan はこのユニットを「状態ユニット」、Elman は対応する構造を「文脈ユニット」と呼ぶ。本稿では「文脈ユニット（context unit）」に統一する。
- 📍 p.12 第1段落・脚注2、p.13 図5

第3段落（Elman ネットワーク）

- Elman（1990）のアーキテクチャは Jordan のものより単純である（図6）。
  - 隠れ層の各ユニットに文脈ユニットが1つずつ対応する。
  - 文脈ユニット j' は、対応する隠れノード j の1時刻前の状態を、重み $w_{j'j} = 1$ で固定した辺を通して受け取る。その値を通常の辺で同じ隠れノード j に戻す。
- これは、各隠れノードが自己結合の再帰辺を1本ずつ持つ単純な RNN と等価である。
- 重みを固定した再帰辺で隠れノードを自己結合させるという考えは、後の LSTM（Hochreiter and Schmidhuber, 1997）の基礎になった。
- 📍 p.12 第2段落、p.14 図6

第4段落（Elman の XOR 実験）

- Elman（1990）は誤差逆伝播法で学習し、ネットワークが時間的な依存関係を学習できることを示した。実験は2種類ある。
- 1つ目は、論理演算 XOR を時間方向に拡張した課題である。
  - 3トークンの区切りを連結した系列を使う。例えば「011」では、最初の2つ（「01」）をランダムに選び、3つ目（「1」）を最初の2つの XOR で決める。
  - ランダムに当てると正解率は50%になる。理想的なシステムは、最初の2トークンではランダムと同じだが、3つ目を必ず当てるので正解率は66.7%になる。
  - Elman の単純なネットワークは、この上限にほぼ到達した。
- 【注】本文は「2種類の実験」と書いているが、紹介しているのは1つ目（XOR）だけである。
- 📍 p.12 第3段落、p.13 第1段落

3.2 再帰型ネットワークの学習

第1段落（勾配消失・勾配爆発の仕組み）

- 再帰型ネットワークの学習は長く難しいとされてきた。順伝播型でさえ最適化は NP 完全である（Blum and Rivest, 1993）。
- 再帰型では特に、長距離の依存関係の学習が難しい（Bengio et al., 1994、Hochreiter et al., 2001）。
- 勾配消失・勾配爆発は、多くの時刻にわたって誤差を逆伝播するときに起こる。
- 簡単な例：入力ノード1つ、出力ノード1つ、再帰する隠れノード1つのネットワーク（図7）を考える。
  - 時刻 τ に入力を与え、時刻 t で誤差を計算する。途中の時刻の入力は0とする。
  - 重みは時刻間で共有されるので、隠れノード j の再帰辺の重みは常に同じ値である。

    - そのため、時刻 τ の入力が時刻 t の出力に与える寄与は、$t-\tau$ が大きくなるにつれて指数的に爆発するか、0に近づく。
    - したがって、入力についての誤差の微分も爆発するか消失する。
- 📍 p.13 §3.2 第1段落

第2段落（どちらが起こるかと Pascanu らの解析）

- 消失と爆発のどちらが起こるかは、再帰辺の重みが $|w_{jj}| > 1$ か $|w_{jj}| < 1$ か、および隠れノードの活性化関数で決まる（図8）。
    - シグモイドでは勾配消失がより深刻になる。
    - ReLU $\max(0, x)$ では勾配爆発のほうが起こりやすい。
- Pascanu et al.（2012）は、勾配消失・爆発が起こる正確な条件を数学的に示した。その条件に基づき、勾配が消失も爆発もしない値に重みを保つ正則化項を使った学習法を提案した。
- 📍 p.14 第1段落

図7・図8（勾配消失の可視化）

- 図7：入力ユニット1つ、出力ユニット1つ、再帰する隠れユニット1つの単純な再帰型ネットワーク。
- 図8：図7のネットワークで勾配消失を可視化したもの（Graves, 2012 を改変）。再帰辺の重みが1より小さいと、最初の時刻の入力が最後の時刻の出力に与える寄与は、時間間隔の長さに対して指数的に減る。
- 📍 p.15 図7、図8

第3段落（打ち切り BPTT と LSTM）

- 打ち切り BPTT（truncated BPTT, TBPTT）は、動き続けるネットワークで勾配爆発を防ぐ方法の1つである（Williams and Zipser, 1989）。
    - 誤差を逆伝播する時刻数に上限を設ける。
    - 上限を小さくすれば勾配爆発を抑えられるが、その代わり長距離の依存関係を学習できなくなる。
- 後述する LSTM は、重み1で固定した再帰辺を持つ、注意深く設計したノードを使って勾配消失を解決する。
- 📍 p.14 第2段落

第4段落（局所最適解の問題）

- 局所最適解は、アーキテクチャを変えるだけでは対処できない学習の障害である。隠れ層1層の順伝播型ネットワークでさえ、最適化は NP 完全である（Blum and Rivest, 1993）。
- しかし最近の実験的・理論的研究によると、実際にはかつて考えられていたほど重要な問題ではない可能性がある。
- Dauphin et al.（2014）は次のことを示した。
    - 大きなニューラルネットワークの誤差曲面には臨界点が多数ある。
    - 臨界点のうち、本当の局所最小値に対する鞍点の比率は、ネットワークの大きさに対して指数的に増える。
    - 鞍点から抜け出すアルゴリズムは設計できる。
- 【注】p.8 §2.3 では厳密な最適化を「NP 困難」、ここでは「NP 完全」と書いており、論文内で表現がそろっていない。
- 📍 p.14 第3段落、p.16 冒頭

第5段落（学習を可能にした要因）

- 後述の改良されたアーキテクチャに加えて、高速な実装とよりよい勾配追従のヒューリスティクスによって、RNN の学習が実用的になった。
- Theano（Bergstra et al., 2010）や Torch（Collobert et al., 2011）のような GPU を使う順伝播・逆伝播の実装によって、高速な学習アルゴリズムを簡単に実装できるようになった。
- LSTM が登場する前の1996年には、長い時間差をつなぐように再帰型ネットワークを学習させても、ランダムな推測と変わらないことが示されていた（Hochreiter and Schmidhuber, 1996）。現在は RNN の学習が成功することが多い。
- 📍 p.16 第1段落

第6段落（二次の最適化手法）

- タスクによっては、無料のソフトウェアを GPU 1台で動かすだけで、数時間で説得力のある結果が得られる（Karpathy, 2015）。
- Martens and Sutskever（2011）は、ヘッセ行列を使わない打ち切りニュートン法（Hessian-free）で RNN の学習に成功した。この手法を、1文字ずつ文章を生成するネットワークに適用した（Sutskever et al., 2011）。
- Dauphin et al.（2014）は、鞍点を避けるニュートン法（saddle-free Newton method）を提案した。
  - 通常のニュートン法は鞍点を含む臨界点に引き寄せられるが、この変種は鞍点から抜け出すように設計されている。
  - 再帰型ネットワークで性能が改善した。
  - ニュートン法はヘッセ行列が必要で、計算量がパラメータ数の2乗で増えるため、大きなネットワークでは計算できない。この手法はヘッセ行列を近似するが、それでも SGD より計算コストが高い。
  - そこで、SGD が停滞していると見られる場所だけでこの手法を使う混合方式を示した。
- 📍 p.16 第2段落

4 現代の RNN アーキテクチャ

第1段落（2つの源流）

- 系列学習で最も成功した RNN アーキテクチャは、1997年に発表された2本の論文に由来する。
- 1本目は Hochreiter and Schmidhuber（1997）の「Long Short-Term Memory」である。
  - 隠れ層の通常のノードを置き換える計算単位として、記憶セル（memory cell）を導入した。
  - 記憶セルによって、以前の再帰型ネットワークが抱えていた学習の困難を克服できる。
- 2本目は Schuster and Paliwal（1997）の「Bidirectional Recurrent Neural Networks」である。
  - 系列の各時点の出力を、過去と未来の両方の情報から決めるアーキテクチャを導入した。
  - 過去の入力しか出力に影響しない以前のネットワークとは対照的である。自然言語処理の系列ラベリングなどで成功している。
- 2つの工夫は両立でき、組み合わせたものが音素分類（Graves and Schmidhuber, 2005）や手書き文字認識（Graves et al., 2009）で成功している。
- この章では LSTM と BRNN を説明し、さらにアドレス指定できる外部メモリで RNN を拡張したニューラルチューリングマシン（NTM, Graves et al., 2014）を紹介する。
- 📍 p.16 §4

4.1 長短期記憶（LSTM）

図9（元の LSTM の記憶セル）

- 自己結合したノードが内部状態 s である。斜線は線形（恒等関数を活性化関数にする）を表す。

- 青い破線は重み1で固定した再帰辺である。Π のノードは入力の積を出力する。Π ノードに出入りする辺もすべて重み1で固定である。
- 📍 p.17 図9

第1段落（LSTM の目的と構造）

- Hochreiter and Schmidhuber（1997）は、主に勾配消失を克服するために LSTM を提案した。
- 隠れ層を持つ標準的な RNN に似ているが、隠れ層の通常のノード（図1）を記憶セル（図9）に置き換える。
- 各記憶セルは、重み1で固定した自己結合の再帰辺を持つノードを含む。これにより、勾配は消失も爆発もせずに多くの時刻を通過できる。
- 通常のノードではなく記憶セルを指すときは、添字 c を使う。
- 📍 p.17 §4.1 第1段落

第2段落（名前の由来）

- 単純な RNN は、重みという形で長期記憶を持つ。重みは学習中にゆっくり変わり、データについての一般的な知識を蓄える。
- また、ノードからノードへ渡る一時的な活性という形で短期記憶を持つ。
- LSTM は、記憶セルによってその中間の記憶を導入する。これが「長短期記憶」という名前の由来である。
- 記憶セルは、単純なノードを特定のパターンで結合した複合ユニットで、乗算ノード（図では Π）を含む点が新しい。
- ベクトル表記は1層のセル全体の値を表す。例えば $\boldsymbol{s}$ は、層の各記憶セル c の s_c を並べたベクトルである。添字 c は個々の記憶セルを指す。
- 📍 p.17 第2段落、p.18 冒頭

LSTM の構成要素

- 入力ノード（input node）g_c
  - 現在の入力層 $\boldsymbol{x}^{(t)}$ と、（再帰辺を通して）1時刻前の隠れ層 $\boldsymbol{h}^{(t-1)}$ から、通常のやり方で活性を受け取る。
  - 重み付き和には通常 tanh を適用する。ただし元の LSTM 論文ではシグモイドである。
- 入力ゲート（input gate）i_c
  - ゲートは LSTM に特有の要素である。入力ノードと同じく $\boldsymbol{x}^{(t)}$ と1時刻前の隠れ層から活性を受け取るシグモイドのユニットである。
  - 値が別のノードの値に掛けられるので「ゲート」と呼ぶ。値が0なら流れを遮断し、1ならすべて通す。
  - 入力ゲートの値 i_c は入力ノードの値に掛けられる。
- 内部状態（internal state）s_c
  - 各記憶セルの中心にある、線形の活性化関数を持つノードである。
  - 重み1で固定した自己結合の再帰辺を持つ。この辺は一定の重みで隣り合う時刻をつなぐので、誤差は消失も爆発もせずに時刻をまたいで流れる。この辺は定誤差カルーセル（constant error carousel）と呼ばれる。
  - 内部状態の更新式は次のとおり。ここで、$\odot$ は要素ごとの積である。

$$\boldsymbol{s}^{(t)} = \boldsymbol{g}^{(t)} \odot \boldsymbol{i}^{(t)} + \boldsymbol{s}^{(t-1)}$$

- 忘却ゲート（forget gate）f_c

    - Gers et al.（2000）が導入した。ネットワークが内部状態の中身を消去することを学習できるようにする。動き続けるネットワークで特に役立つ。
    - 忘却ゲートを使う場合、内部状態の更新式は次のようになる。

$$\boldsymbol{s}^{(t)} = \boldsymbol{g}^{(t)} \odot \boldsymbol{i}^{(t)} + \boldsymbol{f}^{(t)} \odot \boldsymbol{s}^{(t-1)}$$

- 出力ゲート（output gate）o_c
    - 記憶セルが最終的に出力する値 v_c は、内部状態 s_c に出力ゲートの値 o_c を掛けたものである。
    - 内部状態にまず tanh を適用するのが慣例である。各セルの出力が、通常の tanh の隠れユニットと同じ値域になるためである。
    - 一方、他の研究では値域の広い ReLU のほうが学習しやすい。そのため、内部状態にかける非線形関数は省略してもよい可能性がある（著者の見解）。
- 📍 p.18 箇条書き

第3段落（記号の慣例）

- 元の論文とその後の多くの研究では入力ノードを g と書く。本稿もこれに従うが、g は gate の意味ではないので紛らわしい。
- 元の論文ではゲートを y_{in}、y_{out} と書くが、機械学習では y は普通出力を表すので紛らわしい。本稿では Sutskever et al.（2014）にならい、入力・忘却・出力ゲートを i、f、o と書く。
- 📍 p.18 最終段落、p.19 第1段落

図10（忘却ゲート付きの記憶セル）

- Gers et al.（2000）の、忘却ゲートを持つ LSTM の記憶セルである。
- 📍 p.19 図10

第4段落（LSTM の変種とのぞき穴結合）

- 元の LSTM の後、いくつかの変種が提案された。
- 忘却ゲートは2000年に提案され、元の設計には含まれていなかった。効果が確認され、現在のほとんどの実装で標準になっている。
- 同じ年に Gers and Schmidhuber（2000）は、のぞき穴結合（peephole connection）を提案した。
    - 内部状態から、同じセルの入力ゲートと出力ゲートへ、出力ゲートを通さずに直接つなぐ結合である。
    - 事象の間の正確な時間間隔を測る必要があるタイミング課題で性能が上がると報告されている。
- 直感的な例：物体を n 個見たら決まった出力を出す、という数え上げを学習するネットワークを考える。
    - ネットワークは、物体を1つ見るたびに一定量の活性を内部状態に入れることを学習しうる。
    - 活性は定誤差カルーセルによって内部状態 s_c に閉じ込められ、物体を見るたびに増えていく。
    - n 個目を見たとき、出力に影響させるために内部状態の中身を外に出す必要がある。そのためには出力ゲート o_c が s_c の値を知っている必要がある。
    - したがって、s_c を o_c の入力にするべきである。
- 📍 p.19 第2段落

図11（記憶セル2つの隠れ層）

- 隠れ層が記憶セル2つからなる再帰型ネットワークを、2時刻分展開したものである。
- 📍 p.20 図11

第5段落（忘却ゲート付き LSTM の全計算）

- 現代の、忘却ゲートを持つ LSTM の各時刻の計算は次のとおりである。

$$
\begin{aligned}
\boldsymbol{g}^{(t)} &= \phi\left(W^{\mathrm{gx}}\boldsymbol{x}^{(t)} + W^{\mathrm{gh}}\boldsymbol{h}^{(t-1)} + \boldsymbol{b}_g\right) \\
\boldsymbol{i}^{(t)} &= \sigma\left(W^{\mathrm{ix}}\boldsymbol{x}^{(t)} + W^{\mathrm{ih}}\boldsymbol{h}^{(t-1)} + \boldsymbol{b}_i\right) \\
\boldsymbol{f}^{(t)} &= \sigma\left(W^{\mathrm{fx}}\boldsymbol{x}^{(t)} + W^{\mathrm{fh}}\boldsymbol{h}^{(t-1)} + \boldsymbol{b}_f\right) \\
\boldsymbol{o}^{(t)} &= \sigma\left(W^{\mathrm{ox}}\boldsymbol{x}^{(t)} + W^{\mathrm{oh}}\boldsymbol{h}^{(t-1)} + \boldsymbol{b}_o\right) \\
\boldsymbol{s}^{(t)} &= \boldsymbol{g}^{(t)} \odot \boldsymbol{i}^{(t)} + \boldsymbol{s}^{(t-1)} \odot \boldsymbol{f}^{(t)} \\
\boldsymbol{h}^{(t)} &= \phi\left(\boldsymbol{s}^{(t)}\right) \odot \boldsymbol{o}^{(t)}
\end{aligned}
$$

- 記号
  - $\boldsymbol{h}^{(t)}$：時刻 t の LSTM の隠れ層の値
  - $\boldsymbol{h}^{(t-1)}$：1時刻前に隠れ層の各記憶セルが出力した値
  - ϕ：tanh、σ：シグモイド
- この式は忘却ゲートを含むが、のぞき穴結合は含まない。忘却ゲートのない単純な LSTM は、すべての t で $\boldsymbol{f}^{(t)} = 1$ とすれば得られる。
- 入力ノード g には、Zaremba and Sutskever（2014）の最先端の設計にならって tanh を使う。元の LSTM 論文ではシグモイドである。
- 【注】原文 p.20 の $\boldsymbol{s}^{(t)}$ の式は $\boldsymbol{i}^{(i)}$ と印刷されているが、$\boldsymbol{i}^{(t)}$ の誤植である。上の式では直してある。
- 📍 p.19 最終段落、p.20 式と第1段落

第6段落（LSTM の直感的な理解）

- 順伝播で見ると、LSTM は内部状態にいつ活性を入れるかを学習できる。入力ゲートが0の間は何も入らない。同様に、出力ゲートはいつ値を外に出すかを学習する。
- 両方のゲートが閉じていると、活性は記憶セルに閉じ込められる。増えも減りもせず、途中の時刻の出力にも影響しない。
- 逆伝播で見ると、定誤差カルーセルによって、勾配は爆発も消失もせずに多くの時刻をさかのぼれる。この意味で、ゲートは誤差をいつ入れ、いつ出すかを学習している。
- 実際に LSTM は、単純な RNN より長距離の依存関係をうまく学習できる。そのため、本稿で取り上げる最先端の応用研究の多くは LSTM を使う。
- 📍 p.20 最終段落、p.21 第1段落

第7段落（複数の記憶セルで隠れ層を作る方法）

- 複数の記憶セルをどう組み合わせて隠れ層を作るかは、よく誤解される。そこで図4に対応する、記憶セル2つの簡単なネットワークを図11に示す。
- 各記憶セルの出力は、次の時刻に、すべての記憶セルの入力ノードとすべてのゲートに入る。
- 記憶セルの層を複数重ねることも多い（Sutskever et al., 2014）。その場合、各層は同じ時刻の1つ下の層と、1時刻前の同じ層から入力を受けるのが一般的である。
- 📍 p.21 第2段落

4.2 双方向再帰型ニューラルネットワーク（BRNN）

第1段落（BRNN の構造と式）

- LSTM と並んで最もよく使われる RNN アーキテクチャの1つが、Schuster and Paliwal（1997）が提案した BRNN である（図12）。
- 隠れノードの層が2つあり、どちらも入力と出力につながる。
  - 1つ目の層は、過去の時刻からの再帰結合を持つ。
  - 2つ目の層は再帰結合の向きが逆で、系列を後ろ向きに活性を伝える。
- 入力系列と目標系列があれば、時間方向に展開して通常の誤差逆伝播法で学習できる。
- BRNN は次の3式で表される。

$$\boldsymbol{h}^{(t)} = \sigma\left(W^{\mathrm{hx}}\boldsymbol{x}^{(t)} + W^{\mathrm{hh}}\boldsymbol{h}^{(t-1)} + \boldsymbol{b}_h\right)$$
$$\boldsymbol{z}^{(t)} = \sigma\left(W^{\mathrm{zx}}\boldsymbol{x}^{(t)} + W^{\mathrm{zz}}\boldsymbol{z}^{(t+1)} + \boldsymbol{b}_z\right)$$
$$\hat{\boldsymbol{y}}^{(t)} = \mathrm{softmax}\left(W^{\mathrm{yh}}\boldsymbol{h}^{(t)} + W^{\mathrm{yz}}\boldsymbol{z}^{(t)} + \boldsymbol{b}_y\right)$$

- ここで、$\boldsymbol{h}^{(t)}$ は順方向、$\boldsymbol{z}^{(t)}$ は逆方向の隠れ層の値である。$\boldsymbol{z}^{(t)}$ は1時刻**後**の $\boldsymbol{z}^{(t+1)}$ から計算する。
- 📍 p.21 §4.2、p.22 第1段落と式、p.21 図12

第2段落（BRNN の限界と向いている課題）

- BRNN は未来と過去の両方に決まった終点が必要なので、動き続けることができない。
- まだ観測していない未来の要素は受け取れないので、オンライン学習にも向かない。
- しかし長さが決まった系列の予測では、過去と未来の両方の要素を考慮するのが妥当なことが多い。
  - 例：品詞タグ付け。文中のある単語の品詞を予測するには、その前の単語と後の単語の両方の情報が役立つ。
- 📍 p.22 第2段落

第3段落（LSTM と BRNN の組み合わせ）

- LSTM と BRNN は両立する考え方である。LSTM は隠れ層を作る新しい基本単位を導入し、BRNN は中身のノードによらず隠れ層の配線を決める。
- 両者を組み合わせた BLSTM は、手書き文字認識と音素分類で最先端の結果を出している（Graves and Schmidhuber, 2005、Graves et al., 2009）。
- 📍 p.22 第3段落

4.3 ニューラルチューリングマシン

第1段落（NTM の発想）

- ニューラルチューリングマシン（NTM）は、アドレス指定できる外部メモリで RNN を拡張したものである（Graves et al., 2014）。ソートのような複雑なアルゴリズム的課題を解く能力を高める。
- 人間はメモリバッファとやりとりする「中央実行系」を持つという認知科学の理論（Baddeley et al., 1996）に着想を得ている。
- チューリング機械では、プログラムが読み取りヘッドと書き込みヘッドを操作してテープ（外部メモリ）とやりとりする。これになぞらえて NTM と名付けられた。
- 読み書きヘッドの技術的な詳細は本稿の範囲外とし、モデルと応用の大まかな姿だけを示す。
- 📍 p.22 §4.3 第1段落

第2段落（NTM の構成）

- 主な構成要素はコントローラとメモリ行列の2つである。

    - コントローラ：再帰型でも順伝播型でもよい。外界から入力を受けて出力を返し、メモリに命令を送ってメモリから読み出す。
    - メモリ：N 個の記憶場所からなる大きな行列で、各記憶場所は M 次元のベクトルである。
    - 複数の読み取り・書き込みヘッドが、コントローラとメモリ行列のやりとりを仲介する。
- こうした機能を追加しても NTM は端から端まで微分可能で、BPTT を使う SGD の変種で学習できる。
- 📍 p.22 第2段落、p.23 冒頭

第3段落（NTM の実験）

- Graves et al.（2014）は、5つのアルゴリズム的課題で NTM の性能を調べた。アルゴリズム的とは、入力に対する正解出力が、どんな汎用プログラミング言語でも簡単に書ける単純なプログラムで計算できるという意味である。
    - コピー課題：固定長の2値ベクトルの系列と区切り記号を入力し、入力系列のコピーを出力する。
    - 優先度ソート課題：2値ベクトルの系列と、各ベクトルに付いた異なるスカラーの優先度を入力し、優先度順に並べたベクトル系列を出力する。
- 実験では、NTM が教師あり学習によってこれらのアルゴリズムを正しく効率よく実装できるかを調べた。
- この方法で得た解は、訓練データより長い入力にもある程度汎化する。外部メモリのない LSTM は長い入力にうまく汎化しない。
- LSTM RNN、順伝播型コントローラの NTM、LSTM コントローラの NTM の3つを比べると、どの課題でも2つの NTM が訓練データの性能とテストデータへの汎化の両方で LSTM RNN を大きく上回った。
- 📍 p.23 第1段落

5 LSTM と BRNN の応用

第1段落（この章の概要と図13）

- 前章までで、最先端の RNN のほぼすべてを構成する部品を紹介した。この章では、再帰型ネットワークが成功した応用分野を取り上げる。
- 結果を詳しく述べる前に、重要な課題を RNN の系列学習問題としてどう表現するかを、図13で具体的に示す。
- 📍 p.23 §5 第1段落

図13（RNN による入出力の形）

- 青い四角は入力、赤い四角は出力、緑の四角はネットワークの隠れ状態全体を表す（Karpathy, 2015 の図をもとにしている）。
    - (a) 1入力→1出力：標準的な順伝播型が仮定する、独立な場合
    - (b) 系列→1つの固定長ベクトル：文章や動画の分類
    - (c) 1入力→系列：画像キャプション生成。入力画像は系列でない単一のデータ点
    - (d) 系列→系列（長さが異なってよい）：自然言語の翻訳
    - (e) 系列→系列（各時刻に出力）：各ステップで次の文字を予測する、文章の生成モデル
- 📍 p.24 図13

第2段落（この章の構成と評価の難しさ）

- まず RNN の入出力に使う自然言語の表現と、系列予測でよく使う評価指標を紹介する。次に、機械翻訳、画像キャプション生成、動画キャプション生成、手書き文字認識の最先端の結果を概観する。

- RNN の応用の多くは書き言葉を扱う。画像キャプション生成は文字列を生成し、機械翻訳や対話システムは文章の入力と出力の両方を必要とする。
- 文章を出力するシステムは、2値の予測や数値を出力するシステムより実験的な評価が難しい。そのため、翻訳やキャプションの質を評価する方法がいくつも作られてきた。
- 📍 p.23 第2段落

5.1 自然言語の入出力の表現

第1段落（出力の表現）

- 各時刻に単語を出力する場合、出力は普通 softmax のベクトル $\boldsymbol{y}^{(t)} \in \mathbb{R}^K$ である。ここで、K は語彙の大きさである。
- softmax 層は、全成分の和が1になるよう正規化したロジスティック関数を要素ごとに適用したものである。各出力は、その時刻にその単語が正しい出力である確率に対応する。
- 📍 p.25 §5.1 第1段落

第2段落（入力の表現：one-hot と分散表現）

- 入力が単語の系列なら、普通は単語を1時刻に1つずつネットワークに入れる。
- 最も単純な表現は one-hot 符号化である。語彙の大きさと同じ長さの2値ベクトルで、例えば「1000」は語彙の1番目、「0100」は2番目の単語を表す（Elman, 1990 など）。
- しかし one-hot 符号化は非効率で、語彙の数だけビットが必要になる。また、単語間の類似性を符号の中で直接表せない。
- そのため現在は、意味ベクトルを使った分散表現で単語を表すのが一般的である。
  - 大規模な教師ありデータから学習することもあるが、単語の共起統計に基づく埋め込みで初期化するのが普通である。
  - 共起統計から単語ベクトルを作る無料のコードとして、GloVe（Pennington et al., 2014）と word2vec（Mikolov et al., 2013 のアルゴリズムの実装。Goldberg and Levy, 2014 が解説）がある。
- 📍 p.25 第2段落

第3段落（分散表現の歴史と recursive との区別）

- 文章データの分散表現は Hinton（1986）が示し、Bengio et al.（2003）が自然言語で広く使い、最近は再帰的オートエンコーダ（recursive auto-encoder, RAE）の一連の論文（Socher et al., 2010, 2011a–c）で深層学習の分野に広く知られた。
- これらの recursive なネットワークは recurrent なネットワークではない。本稿の RNN は常に recurrent neural network を指す。
- 両者は別の手法だが、重みを広く共有することと、誤差逆伝播法で端から端まで学習することは共通している。
- 📍 p.25 第3段落

第4段落（文字単位の入出力）

- RNN の多くの実験（Elman, 1990、Sutskever et al., 2011、Zaremba and Sutskever, 2014）では、単語ではなく1文字ずつ入力し、1文字ずつ出力する。
- 出力はほぼ常に softmax 層だが、1文字の入力をどう表現したかを書いていない論文が多い。one-hot 符号化だと推測するのが妥当である。
- 1文字単位で分散表現を使った論文を、著者らは知らない。

- 📍 p.25 第4段落

5.2 評価方法

第1段落（評価の難しさ）

- 可変長の単語系列を出力するシステムをうまく学習させるうえで、評価指標の欠陥は深刻な障害になっている。
- キャプション生成や翻訳では、正しい訳が複数ありうる。ラベル付きデータセットにも、1つの例に参照訳が複数含まれることがある。
- このような正解と比べるのは、2値分類の標準的な評価指標を使うより難しい。
- 📍 p.25 §5.2、p.26 冒頭

第2段落（BLEU）

- 参照が複数ある自然言語出力でよく使う指標が BLEU（Papineni et al., 2002）である。
  - 修正ユニグラム適合率に関係する指標で、$n = 1$ から上限 N までの n グラム適合率の幾何平均をとる。
  - 実際には $N = 4$ がよく使われ、人間の評価者との一致が最大になることが示されている。
- 訳を極端に短くすれば適合率を高くできるので、BLEU には短さへのペナルティ B を入れる。c を候補訳の平均の長さ、r を参照訳の平均の長さとすると、

$$B = \begin{cases} 1 & (c > r) \\ e^{(1-r/c)} & (c \leq r) \end{cases}$$

- BLEU スコアは次の式で計算する。

$$\mathrm{BLEU} = B \cdot \exp\left(\frac{1}{N}\sum_{n=1}^{N} \log p_n\right)$$

- ここで、p_n は修正 n グラム適合率である。候補訳の n グラムのうち、いずれかの参照訳に現れるものの数を、候補訳の n グラムの総数で割ったものである。参照が複数ある場合に適合率を合わせたものなので「修正」と呼ぶ。
- 📍 p.26 第1段落

第3段落（BLEU の限界）

- BLEU は最近の論文で翻訳とキャプション生成の両方の評価によく使われ、人間の判断と強く相関するように見える。
- しかし、BLEU が高い訳が低い訳より優れているという保証はない。大量の訳の集合では人間の判断と相関しやすいが、1文単位では人間の判断を正確に予測できない。
- 📍 p.26 第2段落

第4段落（METEOR）

- METEOR（Banerjee and Lavie, 2005）は BLEU の弱点を克服するための指標である。
  - 候補文と参照文の単語どうしの明示的な一致に基づく。参照が複数あれば最もよいスコアを使う。
  - BLEU と違い、既知の同義語と語幹の一致も使う。
- 計算手順

    - まず、単語の一致に基づいて F 値を計算する。ここで、P は適合率、R は再現率である。

$$F_\alpha = \frac{P \cdot R}{\alpha \cdot P + (1-\alpha) \cdot R}$$

- 次に、断片化ペナルティ $M \propto c/m$ を計算する。
    - c：候補と参照の両方で単語が隣り合っている、連続した単語のかたまり（chunk）の最小数
    - m：一致したユニグラムの総数
- 最後に、スコアは次のようになる。

$$\text{METEOR} = (1 - M) \cdot F_\alpha$$

- 【注】BLEU の短さペナルティの c（候補訳の平均の長さ）と、METEOR の c（かたまりの数）は同じ文字で別の量を表している。
- 📍 p.26 第3段落と式

第5段落（METEOR の利点と欠点）

- 実験的には、METEOR は BLEU より人間の評価者とよく一致する。
- しかし BLEU より計算が複雑である。他者の METEOR スコアを再現するには、語幹処理、同義語の一致、計算をすべて同じにする必要がある。
- どちらの指標も、参照訳の集合がまったく同じであることを前提にしている。
- 📍 p.27 第1段落

第6段落（評価指標についての著者の懸念）

- 系列の依存関係のない2値分類でさえ、F1 のようなよく使う指標では、最適な閾値の決め方が「よい性能」の直感と合わないことがある（Lipton et al., 2014）。
- 同様に、上の指標は本当の目的の弱い代理にすぎない。そのため、本当に強いシステムと、使われている指標に過学習しているだけのシステムを区別するのが難しい可能性がある。
- 📍 p.27 第2段落

5.3 自然言語の翻訳

第1段落（翻訳で語順が重要な理由）

- 文章の翻訳は、浅い手法では解けない機械学習の基本問題である。
- 文書分類のように、語順を無視する bag-of-words 表現でうまくいく課題もある。しかし翻訳では語順が本質的である。
    - 例：「Scientist killed by raging virus」と「Virus killed by raging scientist」は、bag-of-words 表現では同じになる。
- 📍 p.27 §5.3 第1段落

第2段落（Sutskever らの sequence to sequence モデル）

- Sutskever et al.（2014）は、多層 LSTM を2つ使う翻訳モデルを提案し、英語からフランス語への翻訳で優れた性能を示した。
    - 1つ目の LSTM で翻訳元の言語の入力文を符号化（encode）し、2つ目の LSTM で翻訳先の言語の出力文を復号（decode）する。
- 手順（図14）

    - 入力文を符号化 LSTM に1単語ずつ入れる。この間は何も出力しない。入力文を逆順に入れると結果が大きく改善することがわかった。
    - 文の終わりに達したら、出力文の開始を表す特別な記号を復号 LSTM に送る。復号 LSTM は、1つ目の LSTM の最終状態も入力として受け取る。2つ目の LSTM は、各時刻に語彙全体の softmax 確率を出力する。
    - 推論時は、ビームサーチで各時刻の分布から最も確からしい単語を選ぶ。文末（EOS）トークンが出るまで2つ目の LSTM を動かす。
- 📍 p.27 第2段落と箇条書き

第3段落（学習と推論）

- 学習では、正しい入力を符号化器に、正しい訳を復号器に入れる。損失は、復号器の出力から sequence to sequence モデル全体に逆伝播する。
- 訓練データの各文について、正しい訳の尤度が最大になるように学習する。
- 推論時は左から右へのビームサーチで出力する単語を決める。各時刻の後、最も確からしい次の単語の候補をいくつか選んで展開する。EOS トークンを出力した時点でビームサーチを終える。
- 学習には、モーメンタムなしの SGD を使った。最初の5エポックの後は、学習率を1エポックに2回半分にした。
- BLEU は 34.81 で、それまでで最良のニューラルネットワークの自然言語処理システムを上回った。ドメイン知識を明示的に組み込んだシステムを含む、ニューラルネットワーク以外の最良の公表結果とも並んだ。
- 別のシステムが出した訳の候補を並べ替える（リランキング）用途に使うと、BLEU は 36.5 になった。
- 📍 p.27 第3段落、p.29 第1段落

図14（sequence to sequence LSTM）

- 符号化モデル（1つ目の LSTM）と復号モデル（2つ目の LSTM）からなる。
- 入力ブロック（青と紫）は単語ベクトルで、対応する隠れ状態に全結合している。赤いノードは softmax 出力である。
- 重みは、符号化の全時刻の間と、復号の全時刻の間でそれぞれ共有している。
- 📍 p.28 図14

第4段落（実装の規模）

- この結果を出した実装は GPU 8台を使い、それでも学習に10日かかった。
    - LSTM の各層に GPU を1台ずつ割り当て、さらに4台を softmax の計算だけに使った。
- C++ で実装し、LSTM の各隠れ層は1,000ノードだった。
- 入力の語彙は160,000語、出力の語彙は80,000語だった。
- 重みは -0.08 から 0.08 の一様乱数で初期化した。
- 📍 p.29 第2段落

第5段落（Auli らの手法と比較できない理由）

- Auli et al.（2013）も RNN による翻訳手法を示した。
    - Mikolov の単語埋め込みと、復号器の出力のラティス表現を使って、ありうる訳の空間を探索しやすくした。ラティスの各ノードは単語の系列に対応する。
    - フランス語から英語への翻訳で BLEU 28.5 を報告している。

- 2つの論文は似たデータセットで結果を出しているが、Sutskever et al. は英語→フランス語、Auli et al. はフランス語→英語しか報告していない。そのため2つのモデルの性能は比較できない。
- 📍 p.29 第3段落

5.4 画像キャプション生成

第1段落（課題の設定）

- 最近、写真を説明する文の生成に RNN が使われ、成功している（Vinyals et al., 2015、Karpathy and Fei-Fei, 2014、Mao et al., 2014）。
- 訓練データは入力画像 $\boldsymbol{x}$ と目標のキャプション $\boldsymbol{y}$ の組で、大量の組から画像に合うキャプションを予測するモデルを学習する。
- 📍 p.29 §5.4 第1段落

第2段落（Vinyals らと Mao ら）

- Vinyals et al.（2015）は、言語から言語への翻訳の成功を受けて、キャプション生成を画像から言語への翻訳とみなした。
  - 符号化と復号の両方を LSTM で行うのではなく、畳み込みニューラルネットワークで画像を符号化し、LSTM で復号する方法を導入した。
- Mao et al.（2014）は独立に似た RNN のキャプション生成ネットワークを作り、Pascal、Flickr30K、COCO の各データセットで当時の最先端の結果を出した。
- 📍 p.29 第2段落

第3段落（Karpathy and Fei-Fei）

- Karpathy and Fei-Fei（2014）はこれらの研究を発展させた。
  - 畳み込みネットワークで画像を符号化し、双方向ネットワークの注意機構と標準的な RNN でキャプションを復号する。単語の表現には word2vec の埋め込みを使う。
  - 画像全体のキャプション生成と、画像の領域と文の断片の対応をとるモデルの両方を扱う。
- 推論時は Sutskever et al.（2014）と同様に、1単語ずつ文を復号する。最も確からしい単語を選んで次の時刻に入れ、EOS トークンが出るまで繰り返す。
- 📍 p.29 第3段落、p.30 冒頭

第4段落（データの規模と改訂版の知見）

- 問題の規模の目安として、Karpathy and Fei-Fei（2014）は3つのデータセットを使った。

データセット	容量	画像数
Flickr8K	50 MB	8,000枚
Flickr30K	200 MB	30,000枚
COCO	750 MB	328,000枚

- 実装には Caffe ライブラリ（Jia et al., 2014）を使い、畳み込みネットワークは ImageNet のデータで事前学習した。
- 改訂版では、次のことを報告している。
  - LSTM は単純な RNN より性能がよい。
  - 単語表現は、word2vec の埋め込みよりランダムな初期値から学習したほうがよいことが多い。

    - 理由として、word2vec は色を表す単語などを埋め込み空間で近くに集めてしまい、画像の視覚的な説明には向かない可能性があると述べている。
- 📍 p.30 第1段落

5.5 その他の応用

第1段落（手書き文字認識）

- 手書き文字認識は、双方向 LSTM が最先端の結果を出した応用分野である。
- Liwicki et al.（2007）と Graves et al.（2009）は、電子ホワイトボードからデータを集めた。センサーが、一定間隔の時刻ごとにペンの (x, y) 座標を記録する。
- 後の論文では双方向 LSTM を使い、単語単位の正解率で 81.5% を達成した。HMM の 70.1% を上回る。
- 📍 p.30 §5.5 第1段落

第2段落（動画への応用）

- この1年で、翻訳や画像キャプション生成での成功を新しい分野に広げる論文がいくつも出ている。
- 特に興味深いのは、教師なしの動画の符号化（Srivastava et al., 2015）、動画のキャプション生成（Venugopalan et al., 2015）、プログラムの実行（Zaremba and Sutskever, 2014）である。
- Venugopalan et al.（2015）は、動画のフレームを符号化して単語を復号する sequence to sequence のアーキテクチャを示した。
    - 符号化 LSTM の各時刻の入力は、畳み込みニューラルネットワークの最上位の隠れ層である。
    - 復号時は、各時刻に語彙全体の確率を出力する。
- 📍 p.30 第2段落

第3段落（プログラムの実行）

- Zaremba and Sutskever（2014）は、コンピュータのプログラムを1文字ずつ読み、その出力を予測するネットワークを実験した。
- 整数を出力するプログラムを対象にした。9桁の数2つの足し算のような単純なプログラムでは、99% の正解率で出力を予測できた。
    - ネットワークは LSTM セルの隠れ層を複数重ね、プログラムを左から右へ1回だけ読む。
- 📍 p.30 第3段落

6 考察

第1段落（30年の変化）

- 過去30年で、RNN は主に認知モデリングや計算論的神経科学の関心の対象だったモデルから、系列の大規模な教師あり学習のための強力で実用的な道具になった。
- この進歩は、モデルのアーキテクチャ、学習アルゴリズム、並列計算の進歩による。
- RNN が特に興味深いのは、従来の機械学習が入力と出力に課してきた多くの制約を克服するからである。RNN では、連続する例の間の独立性の仮定が崩れ、それに伴って入出力の次元が固定という仮定も崩れる。
- 📍 p.30 §6、p.31 第1段落

第2段落（進歩の源泉とモデル探索の自動化）

- LSTM と BRNN は近年多くの課題で正解率の記録を出したが、その進歩は根本的に新しいアルゴリズムではなく、新しいアーキテクチャから生まれている。
- そのため、遺伝的アルゴリズムやマルコフ連鎖モンテカルロ法などで、ありうるモデルの空間の探索を自動化するのは有望かもしれない。
- ニューラルネットワークの技術（新しい活性化関数、学習手順、初期化手順など）は、ネットワークや課題をまたいで移せることが多く、似た効果をもたらす。技術が増えるほど、すべての組み合わせを試すのは現実的でなくなる。
- 著者らの推論：研究者コミュニティ全体が、遺伝的アルゴリズムのように技術を組み合わせ、主要なデータセットの評価指標を適応度関数として、モデルのアーキテクチャと設定の空間を探索している。
- 📍 p.31 第2段落

第3段落（推論から導かれる2つの帰結）

- 1つ目：前段落で述べたとおり、この分野はモデル空間を探索する自動化された手順から恩恵を受けうる。
- 2つ目：より複雑な課題を解くシステムを作るには、よりよい適応度関数（評価指標）が必要である。BLEU スコアは、2値分類の正解率ほど信頼できない。
- そのため可能なら、新しい技術はまず確立したベンチマークのあるデータセットで古典的な順伝播型ネットワークを使って個別に試し、その後で評価基準の信頼性が低い設定の RNN に適用するのが賢明である。
- 📍 p.31 第3段落

第4段落（今後の展望）

- 自然言語の課題で RNN が急速に成功していることから、より長い文章への拡張は実りが多いと著者らは考えている。
- 対話システムも、翻訳と同じ原理のアーキテクチャで作れると予想している。質問を符号化して応答を生成し、会話の履歴全体を文脈として保持する。
- 📍 p.31 第4段落